

A Friendly Primer: Transliteration, Indian Scripts & the AI Behind It

Project: मोडी ते मराठी (modi-to-Marathi) **Repository:** <https://github.com/lgtkgvtv/ai4-transliteration-modi-to-marathi> **Written for:** anyone joining this project — no AI background needed

How to read this: every idea is explained in plain language first, with a real-world picture. Technical words are introduced **in bold** and defined right away. There's a quick-reference glossary at the end. If you only read one section, read Part 1.

Part 1 — The biggest idea: a language is not the same as a script

This one idea clears up almost all the confusion.

- A **language** is the spoken thing — the words, the sounds, the grammar. Marathi, Hindi, and Tamil are languages.
- A **script** is the set of shapes you use to write a language down — the letters.

Think of a **song**. The song (the tune and the words) is the language. The way you write it on paper — neat print, fancy cursive, bubble letters — is the script. It's the same song no matter how you write it.

Two surprising facts fall out of this:

1. **One language can be written in many scripts.** Marathi can be written in the old **Modi** script or the modern **Devanagari** script. Same words, same sounds — different letter shapes.
2. **One script can write many languages.** **Devanagari** is used to write Hindi, Marathi, Sanskrit, Nepali, and more. Same letters, different languages.

LANGUAGE (the words & sounds)

SCRIPT (the letter shapes)

Marathi -----> can be written in Modi
 \-----> or in Devanagari

Devanagari (one script) -----> writes Hindi, Marathi, Sanskrit...

Part 2 — Three words people mix up

Word	What changes	Everyday example
Translation	The language (the meaning)	Marathi "पाणी" → English "water"
Transliteration	The script (the letters), meaning stays	Modi "पाणी" → Devanagari "पाणी" (still the Marathi word for water)
Transcription	Spoken sound → written text	Someone says "paani", you write it down

The key one for us is **transliteration**: we are **not** changing what the words mean. We are rewriting the same Marathi words from old Modi letters into modern Devanagari letters — like retyping your grandmother's cursive diary into clear printed letters so you can read it. The story is unchanged; only the handwriting becomes readable.

Part 3 — Meet the two scripts

Modi script (मोडी): an old, fast, cursive way of writing Marathi, used for many centuries (into the early 1900s), especially for letters and record-keeping — land records, accounts, administration. It was built for speed: written quickly, often with a continuous top line and **few or no spaces between words**.

Devanagari (देवनागरी): the clear, "printed-looking" script most people read Marathi and Hindi in today. If you've seen Hindi on a sign or in a textbook, you've seen Devanagari.

Because schools stopped teaching Modi, **very few people can read it now** — even though an estimated **tens of millions** of Modi documents still exist in archives. Our project is about using a computer to help turn that hard-to-read Modi into easy-to-read Devanagari.

Part 4 — Why this is genuinely hard

If it were easy, it'd be done already. The tricky parts:

- **Look-alike letters.** Some Modi letters look almost identical (like how a quick handwritten a and u can be confused). Examples that trip up both humans and computers: भ/म, क/फ, ट/ठ/ढ, न/ण.
- **No spaces.** Modi often runs words together, so the computer must guess where each word ends.
- **Tiny but important marks.** A small dot or a vowel sign can change the whole sound. Miss the dot and you get the wrong word.
- **Faded, torn, stained pages.** The documents are old. The ones that most need saving are often in the worst shape.
- **Different "handwritings" across time.** Modi from different eras and writers looks different, like comparing a 1700s letter to a 1900s one.

Part 5 — How a computer learns to "read" and "rewrite" (AI, explained simply)

What is an AI "model"?

Normally we tell computers exact rules. But for messy things like handwriting, rules don't work well. Instead we use **Machine Learning (ML)**: we show the computer **lots of examples** and it figures out the patterns itself. The result of all that learning is called a **model** — think of it as the "trained brain" that we can then use.

- **AI (Artificial Intelligence):** the broad idea of computers doing things that seem smart.
- **ML (Machine Learning):** the main way we build AI today — learning from examples.

Training, datasets, and labels

- A **dataset** is the big pile of examples we learn from.
- Each example usually has a **label** — the correct answer. For us: a Modi image (the question) + its correct Devanagari text (the answer).

- **Training** is the practice phase, where the model studies examples and slowly gets better — like a student doing flashcards.

Study set, practice quiz, final exam (and cheating)

We split our data into three piles: - **Training set** — the study material. - **Validation set** — a practice quiz to check progress while learning. - **Test set** — the final exam, kept secret until the end.

Data leakage is accidental cheating — when exam answers sneak into the study material, so the score looks great but the model didn't really learn. Keeping these piles strictly separate is a big deal (our reference team had 10 people manually check for this!).

Reading images: OCR

- **OCR (Optical Character Recognition)**: teaching a computer to find and read letters inside a picture. It's how your phone can copy text out of a photo. Reading handwritten, cursive Modi is a very hard kind of OCR.

The machinery inside (light touch — you don't need the math)

- **Neural network**: the most common kind of "learning machine," loosely inspired by how brains connect. It's layers of simple math that, together, learn complex patterns.
- **Transformer**: today's most popular neural-network design. Almost every modern AI (including ChatGPT-style systems) is a Transformer.
- **Attention**: the trick that makes Transformers good — the model learns to focus on the parts that matter (like focusing on the key word in a sentence).
- **Tokens & tokenizer**: computers don't read whole sentences at once; a **tokenizer** chops text into small pieces called **tokens** (like Lego bricks — sometimes a word, sometimes part of a word). The model works brick by brick.
- **Embedding**: turning a token or an image into a **list of numbers** that captures its meaning, so the computer can compare and combine things. (Words with similar meaning get similar numbers.)
- **Encoder & decoder**: the **encoder** reads and understands the input; the **decoder** writes the output. For us: an encoder understands the Modi image, a decoder writes the Devanagari.
- **Vision encoder**: the special encoder that turns a picture into numbers. Common ones you'll hear named: **CLIP**, **SigLIP**, **ViT** (Vision Transformer).

Two model "types" you'll hear about

- **LLM (Large Language Model):** a model trained on enormous amounts of text that's very good with language. (Examples: LLaMA, Gemma.)
- **VLM (Vision-Language Model):** an LLM that also has eyes — it can look at an image **and** produce text about it. This is what our reference project used, because the task is "look at Modi image → write Devanagari text."

Making big models usable

- **Parameters:** the "knobs" inside a model that get tuned during training. More knobs = bigger, smarter, but heavier. You'll see sizes like **429M** (429 million) or **12B** (12 billion).
- **Fine-tuning:** taking a model that already knows a lot and giving it extra lessons on your specific task. Much cheaper than teaching from zero.
- **LoRA (Low-Rank Adaptation):** a clever, cheap way to fine-tune. Instead of rewriting the whole giant textbook, you add small **sticky notes** with adjustments. Fast, light, and you can share just the sticky notes.
- **Knowledge Distillation (teacher → student):** a big, slow, smart **teacher** model trains a small, fast **student** model to copy its answers. You get something pocket-sized that's almost as good. (Our reference project's small "student" was 429M; the released "teacher" is 12B.)
- **Quantization (e.g., 4-bit):** shrinking a model by storing its numbers more compactly — like saving a photo at smaller file size. It lets a big model fit on a normal desktop computer.
- **Inference:** actually using the trained model to get an answer (as opposed to training it). When you feed in a Modi image and get Devanagari out, that's inference.

A few more must-know words

- **Synthetic data:** made-up training examples we generate ourselves — e.g., taking known Devanagari text and rendering it as Modi images using a Modi **font**. Cheap way to create lots of practice material.
- **Hallucination:** when an AI confidently produces something that looks right but is wrong. Dangerous for historical records, which is exactly why a human must check the output.

- **Human-in-the-loop:** a person reviews and corrects the AI. The AI is an assistant that drafts, not a boss that decides. This is the safe and proven way to use AI for important documents.

Part 6 — How we measure if the AI is any good

You can't improve what you can't measure. These are the "report card" scores:

Term	Plain meaning	Better =
Benchmark	A standard test everyone uses, so models can be compared fairly	—
BLEU	Checks how much the AI's output overlaps with the correct answer. Born for translation.	Higher
CER (Character Error Rate)	What fraction of characters came out wrong	Lower
WER (Word Error Rate)	What fraction of words came out wrong	Lower
ICDAR	A big yearly conference/competition for document reading (where this kind of work gets judged)	—

For transliteration specifically, **CER is often the most honest score**, because we care about getting each character right — a single wrong letter can change a name or a date.

Part 7 — The "pipeline": the assembly line

A **pipeline** is just the assembly line of steps that takes a raw scan and turns it into a finished, checked transliteration.

```
[ Modi image ]
|
v
```

```

[ Clean up the image ]      <- straighten, remove stains, boost faded ink ("preprocessing")
    |
    v
[ AI model reads it ]      <- the VLM does its best draft ("inference")
    |
    v
[ Tidy the output ]        <- fix spacing/formatting ("post-processing")
    |
    v
[ Expert checks & corrects ] <- HUMAN-IN-THE-LOOP
    |
    +--> corrections saved as NEW training examples (the model keeps improving)
    |
    v
[ Finished Devanagari text ]

```

The two steps people forget are the first (cleaning the image) and the last two (expert checking + feeding corrections back). Those are where quality actually comes from.

Part 8 — The "who's who": models, datasets & tools to know

You'll see these names in our docs and around the web. Here's what each one is, in one line.

Datasets (piles of examples): | Name | What it is | |---|---| | **MoDeTrans** | ~2,000 real Modi documents + their Devanagari answers (closest to our task) | | **SynthMoDe** | Synthetic (computer-generated) Modi images for practice | | **Aksharantar** | A huge open dataset (26M pairs) for Indian-language transliteration | | **Dakshina** | A well-known benchmark dataset for Roman ↔ Indian-script transliteration |

Models & systems: | Name | What it is | |---|---| | **MoScNet** | The Modi→Devanagari model from the IIT Roorkee project | | **IndicXlit** | A tiny, open model that transliterates 21 Indian languages (proves small can win) | | **Ithaca** | A famous AI that helps historians restore ancient Greek text (the gold standard for "AI + expert") | | **LLaMA / Gemma** | Big general-purpose language models that others build on top of |

Tools & places (the workshop): | Name | What it is | |---|---| | **Hugging Face** | A website to share and download AI models and datasets — "GitHub for AI" | | **PyTorch** | The most common toolkit (library) for building and running AI models | | **Transformers** (by Hugging Face) | A library that makes it easy to load and run modern models | | **Tesseract** | A popular open OCR tool for reading printed text | | **OpenCV** | A popular toolkit for cleaning up and editing images | | **Unicode** | The worldwide agreement that gives every character a number, so computers everywhere agree on what "ॐ" or a Modi letter is. (Both Devanagari and Modi have official Unicode.) | | **AI4Bharat / Bhashini** | India's big efforts to build open language AI for Indian languages |

Quick acronym cheat-sheet: AI = Artificial Intelligence · ML = Machine Learning · OCR = Optical Character Recognition · LLM = Large Language Model · VLM = Vision-Language Model · LoRA = Low-Rank Adaptation · BLEU / CER / WER = scoring methods · ICDAR = a document-AI conference.

Part 9 — One-look glossary

Term	Kid-friendly meaning
Language	The words & sounds (the song)
Script	The letter shapes (the handwriting)
Translation	Change the meaning into another language
Transliteration	Rewrite the same words in different letters
Transcription	Write down what you hear
Model	The "trained brain" that gives answers
Dataset	The pile of examples it learns from
Label	The correct answer for an example
Training	The practice/study phase
Test set	The secret final exam
Data leakage	Accidental cheating (exam answers in the study pile)
OCR	Computer reading letters in a picture

Term	Kid-friendly meaning
VLM	A model that can see pictures and write text
LLM	A model that's great with language
Token	A small Lego-brick piece of text
Embedding	Turning something into meaning-numbers
Fine-tuning	Extra lessons on your specific task
LoRA	Cheap fine-tuning with "sticky notes"
Distillation	Big teacher trains a small student
Parameters	The model's tunable knobs (more = bigger)
Quantization	Shrinking a model to fit smaller computers
Inference	Actually using the model to get answers
Synthetic data	Made-up practice examples we generate
Hallucination	AI confidently saying something wrong
Human-in-the-loop	A person checks & corrects the AI
Benchmark	A shared, fair test to compare models
Pipeline	The assembly line from input to output

Part 10 — How this maps to our project

Putting it all together for **मोडी ते मराठी (modi-to-Marathi)**:

We are building a **pipeline** that takes **images** of **Modi-script** Marathi, uses a **VLM** to produce a **draft transliteration** in **Devanagari**, and has a **human expert check it** — with every correction saved as new **labeled data** to make the next version better. We'll build a small, trustworthy **test set** first, measure quality with **CER**, lean on **synthetic data** (Modi fonts) to bulk up training cheaply, and stay **human-in-the-loop** the whole way.

That's the entire project, in one paragraph you can now fully understand. Welcome aboard.